Vivado krok po kroku: Z1 i Z2

UCiSW 1 · płyta AUP-ZU3 · instrukcja dla osoby, która pierwszy raz otwiera Vivado

Z1: układ 4-bitowy Y = f(X), symulacja, przełączniki → diody · **Z2:** wynik Z1 na wyświetlaczu OLED

Jak czytać tę instrukcję

- Rób kroki **po kolei**. Niczego nie pomijaj, nawet jeśli wygląda na drobiazg.
- **Szary prostokąt** = coś, co klikasz w Vivado (przycisk, menu, zakładka). `Ctrl`+`S` = skrót klawiszowy.
- **„PPM”** = kliknij **prawym** przyciskiem myszy. **„2×klik”** = kliknij dwa razy lewym.
- Nie masz plików przy sobie? Cały kod do wpisania jest w tej instrukcji.
- Ramka **✔ SPRAWDŹ** mówi, co powinieneś zobaczyć. Jeśli widzisz coś innego, zatrzymaj się i zajrzyj do rozdziału **„Coś nie działa”** na końcu.

Słowniczek (6 słów, które musisz znać)

Słowo	Co to znaczy po ludzku
Projekt	Folder, w którym Vivado trzyma wszystkie Twoje pliki dla jednego zadania.
Plik źródłowy (.vhd)	Kod VHDL opisujący układ. Tu wpisujesz równania.
Testbench	Osobny plik VHDL tylko do symulacji. „Kręci przełącznikami” za Ciebie i pokazuje wykres.
XDC (constraints)	Plik mówiący, który sygnał z kodu idzie do którego fizycznego pinu (przełącznik, dioda, OLED).
Bitstream (.bit)	Gotowy plik wgrywany do płyty. Powstaje z kodu + XDC po kliknięciu **Generate Bitstream**.
Top	Najwyższy plik w hierarchii — od niego Vivado zaczyna budowanie albo symulację.

Co mieć ze sobą na zajęciach

Zadanie 1 (folder `Z1/`)

- `Z1_a.vhd` … `Z1_d.vhd` — kod układu (weź swój wariant)
- `Z1_tb.vhd` — testbench
- `Z1.xdc` — piny

Zadanie 2 (folder `Z2/` + pliki od prowadzącego)

- `OLED_1x32.vhd`, `ZU3.xdc`, `ZU3_Exp.xdc`
- `Z2_BD.xdc` — piny do wersji z diagramem
- `Z2_Top.vhd`, `Z2_tb.vhd`, `Z2.xdc` — wersja „gotowy kod”

Spis treści

1. **Zadanie 1** — projekt, kod, symulacja, piny, płyta, symulacja czasowa
2. **Zadanie 2, sposób A** — diagram blokowy z OLED (tak jak w instrukcji prowadzącego)
3. **Zadanie 2, sposób B** — gotowy kod VHDL z trybem AUTO (X zmienia się sam)
4. **Coś nie działa** — najczęstsze problemy i co z nimi zrobić

Zadanie 1 · układ Y = f(X)

Cel: 4-bitowe X z przełączników → układ → 4-bitowe Y na diodach. Plus dwie symulacje.

1 Uruchom Vivado i zacznij nowy projekt

1. Menu Start → wpisz **Vivado** → uruchom (start trwa chwilę).
2. Na ekranie startowym kliknij **Create Project**. Pojawi się kreator — w pierwszym oknie kliknij **Next**.

2 Nazwa i miejsce projektu

- **Project name:** `Z1`
- **Project location:** `C:/AMD_Proj` (zalecany folder w laboratorium)
- Zostaw zaznaczone ☑ **Create project subdirectory** → **Next**

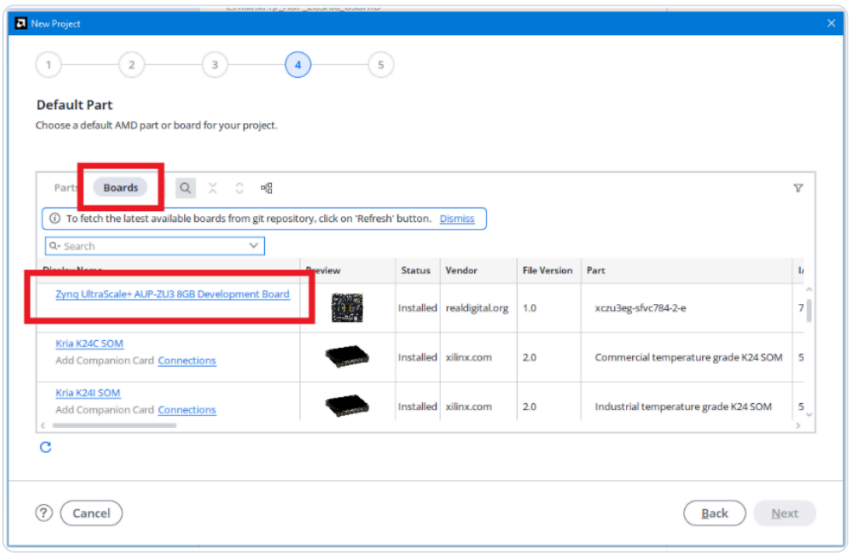
> ⚠ **Bez polskich znaków i spacji**
> w nazwie projektu i w ścieżce (np. nie „Zadanie 1”, nie „ąę”).

3 Typ projektu

- Wybierz **RTL Project**.
- Zaznacz ☑ **Do not specify sources at this time** (pliki dodamy później) → **Next**

4 Wybór płyty AUP-ZU3

1. U góry kliknij zakładkę **Boards** (nie „Parts”).
2. W polu Search wpisz `AUP`.
3. Kliknij wiersz **Zynq UltraScale+ AUP-ZU3 8GB Development Board** → **Next** → **Finish**.

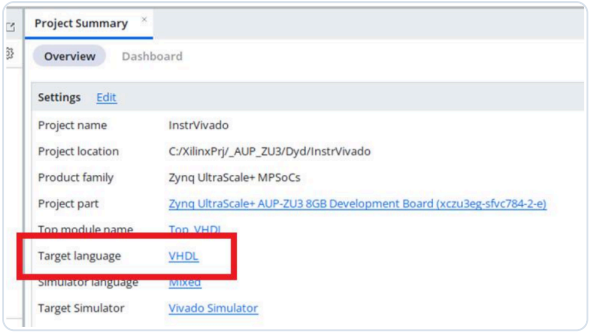


Zakładka Boards i płyta AUP-ZU3 8GB (zrzut z instrukcji prowadzącego)

5 Sprawdź język projektu

Otworzy się główne okno Vivado. W środku jest **Project Summary**. Znajdź **Target language**.

✔ SPRAWDŹ Target language = **VHDL**. Jeśli jest Verilog: kliknij na to słowo i zmień na VHDL.



Project Summary → Target language: VHDL

Mapa okna Vivado

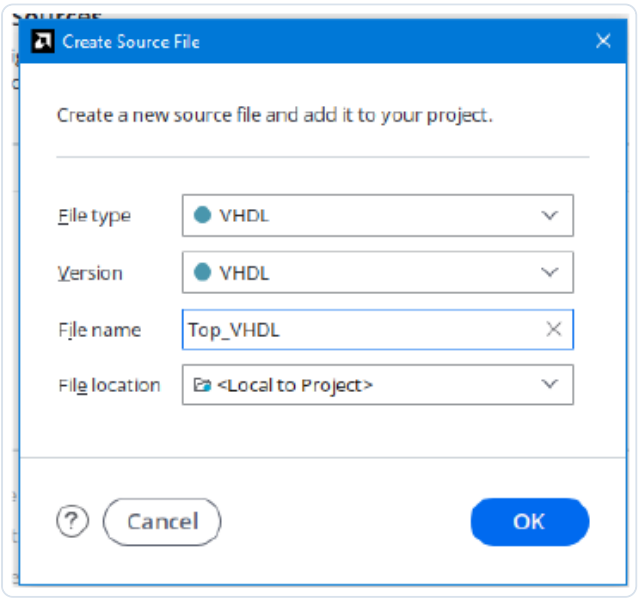
- **Lewa kolumna — FLOW NAVIGATOR:** wszystkie główne polecenia (dodawanie plików, symulacja, bitstream, programowanie). Będziesz tam klikać najczęściej.
- **Okno Sources:** lista Twoich plików.
- **Dół — Tcl Console / Messages:** komunikaty i błędy.

6 Dodaj plik z układem (Z1)

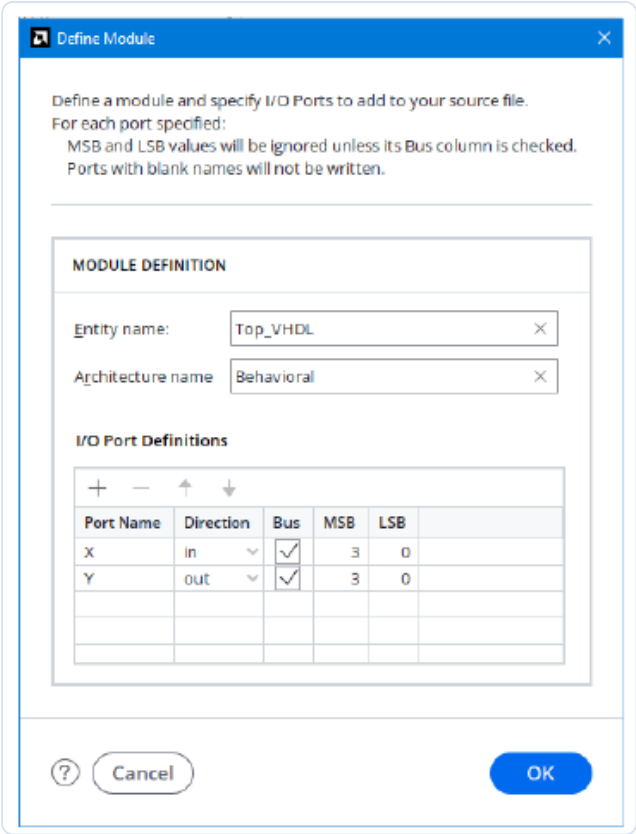
1. Flow Navigator → PROJECT MANAGER → **Add Sources** (albo `Alt`+`A`).
2. Wybierz ◉ **Add or create design sources** → **Next**.
3. Kliknij **Create File**. Ustaw: File type = **VHDL**, File name = `Z1` → **OK** → **Finish**.
4. Otworzy się okno **Define Module** (porty). Wypełnij tabelkę dokładnie tak:

Port Name	Direction	Bus	MSB	LSB
X	in	☑	3	0
Y	out	☑	3	0

→ **OK**.



Create Source File: VHDL + nazwa



Define Module: porty X (in) i Y (out), Bus 3..0

✔ SPRAWDŹ W oknie Sources → Design Sources jest **Z1** (z ikonką „top" obok).

7 Wpisz równania

1. W Sources kliknij 2× na **Z1** — otworzy się edytor z gotowym szkieletem kodu.
2. Na dole pliku znajdź linie `begin` i `end Behavioral;`. **Między nimi** wpisz 4 linie Twojego wariantu (poniżej).
3. Zapisz: `Ctrl`+`S`.

a) Y = (X + 1) mod 16

```
    Y(0) <= not X(0);
    Y(1) <= (X(1) and not X(0)) or (not
X(1) and X(0));
    Y(2) <= (X(2) and not X(1)) or (X(2)
and not X(0))
        or (not X(2) and X(1) and X(0));
    Y(3) <= (X(3) and not X(2)) or (X(3)
and not X(1)) or (X(3) and not X(0))
        or (not X(3) and X(2) and X(1) and
X(0));
```

b) Y = (X + 3) mod 16

```
    Y(0) <= not X(0);
    Y(1) <= (X(1) and X(0)) or (not X(1)
and not X(0));
    Y(2) <= (X(2) and not X(1) and not
X(0))
        or (not X(2) and X(1)) or (not
X(2) and X(0));
    Y(3) <= (X(3) and not X(2)) or (X(3)
and not X(1) and not X(0))
        or (not X(3) and X(2) and X(1)) or
(not X(3) and X(2) and X(0));
```

c) Y = (X – 1) mod 16

d) Y = (X – 3) mod 16

```
    Y(0) <= not X(0);
    Y(1) <= (X(1) and X(0)) or (not X(1)
and not X(0));
    Y(2) <= (X(2) and X(1)) or (X(2) and
X(0))
         or (not X(2) and not X(1) and not
X(0));
    Y(3) <= (X(3) and X(2)) or (X(3) and
X(1)) or (X(3) and X(0))
         or (not X(3) and not X(2) and not
X(1) and not X(0));
```

```
    Y(0) <= not X(0);
    Y(1) <= (X(1) and not X(0)) or (not
X(1) and X(0));
    Y(2) <= (X(2) and X(1) and X(0))
         or (not X(2) and not X(1)) or (not
X(2) and not X(0));
    Y(3) <= (X(3) and X(2)) or (X(3) and
X(1) and X(0))
         or (not X(3) and not X(2) and not
X(1)) or (not X(3) and not X(2) and not
X(0));
```

✔ SPRAWDŹ Na prawym brzegu edytora, u góry paska, jest **zielony kwadracik** = brak błędów. Czerwona kreska = błąd: najedź na nią myszką, przeczytaj komunikat, popraw.

💡 **Masz plik Z1_x.vhd na pendrivie?**
Zamiast kroków 6–7: **Add Sources** → design sources → **Add Files** → wybierz swój `Z1_x.vhd` → zaznacz ☑ **Copy sources into project** → **Finish**.

8 Dodaj testbench (plik do symulacji)

1. **Add Sources** → ◉ **Add or create simulation sources** → **Next**.
2. **Create File** → VHDL, nazwa `Z1_tb` → **OK** → **Finish**.
3. W Define Module **nie dodawaj żadnych portów** → **OK** → na pytanie „are you sure…" kliknij **Yes**.
4. W Sources rozwiń **Simulation Sources** → **sim_1**, kliknij 2× na **Z1_tb**.
5. Zaznacz cały tekst (**Ctrl**+**A**), usuń i wpisz / wklej kod poniżej → **Ctrl**+**S**.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Z1_tb is
end Z1_tb;

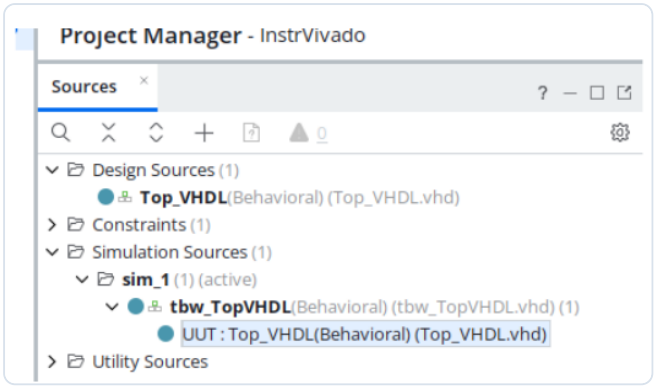
architecture Behavioral of Z1_tb is
    component Z1
        Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
               Y : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

    signal X : STD_LOGIC_VECTOR (3 downto 0);
    signal Y : STD_LOGIC_VECTOR (3 downto 0);
begin
    UUT: Z1 port map ( X => X, Y => Y );

    X <= "0000",
         "0001" after  100 ns, "0010" after  200 ns, "0011" after  300 ns,
         "0100" after  400 ns, "0101" after  500 ns, "0110" after  600 ns,
         "0111" after  700 ns, "1000" after  800 ns, "1001" after  900 ns,
         "1010" after 1000 ns, "1011" after 1100 ns, "1100" after 1200 ns,
         "1101" after 1300 ns, "1110" after 1400 ns, "1111" after 1500 ns;
end Behavioral;
```

Testbench ustawia X = 0, 1, 2 … 15, zmieniając wartość co 100 ns — dokładnie tak, jak każe polecenie. Pełna wersja z automatycznym sprawdzaniem wyników jest w pliku `Z1_tb.vhd` (można dodać przez **Add Files**).

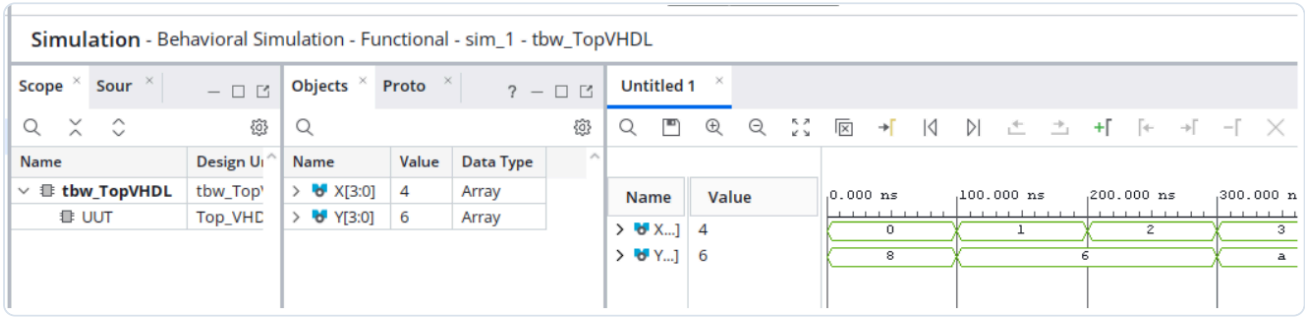
✔ SPRAWDŹ W Simulation Sources: **Z1_tb** jest na górze (pogrubiony), a pod nim **UUT : Z1**. Jeśli Z1_tb nie jest pogrubiony: PPM na nim → **Set as Top**.



Tak ma wyglądać hierarchia: testbench na górze, pod nim UUT

9 Symulacja behawioralna (sprawdzenie logiki)

1. Flow Navigator → SIMULATION → **Run Simulation** → **Run Behavioral Simulation**.
2. Poczekaj — otworzy się okno z wykresem (Waveform).
3. **Ważne:** domyślnie symulacja trwa tylko 1000 ns, a test potrzebuje 1600 ns. Na dole kliknij zakładkę **Tcl Console**, wpisz `run 600 ns` i naciśnij **Enter**.
4. Kliknij ikonę **Zoom Fit** (⛶) nad wykresem — zobaczysz cały przebieg.
5. Żeby widzieć liczby zamiast bitów: PPM na **X** → **Radix** → **Unsigned Decimal**. To samo dla **Y**.



Okno symulacji: po lewej Scope/Objects, po prawej wykres X i Y

✔ SPRAWDŹ X rośnie 0, 1, 2 … 15 co 100 ns, a Y = Twój wzór. Np. dla wariantu a): pod X = 5 jest Y = 6, pod X = 15 jest Y = 0.

💡 **Przydatne przyciski w symulacji**

- **Relaunch** (⟳) — po poprawce w kodzie odświeża symulację bez zamykania okna.
- **Restart** (|◁) — cofa do czasu 0. Potem wpisz np. `run 1600 ns` .
- Powrót do edycji: kliknij **PROJECT MANAGER** na górze Flow Navigatora.

⚠ **Pokaż to prowadzącemu**

przed przejściem dalej — to jest punkt „symulacja behawioralna” z polecenia.

10 Plik XDC — połącz X z przełącznikami i Y z diodami

Najpierw wróć do edycji: kliknij **PROJECT MANAGER** w Flow Navigatorze.

Sposób łatwy (masz plik `Z1.xdc`): Add Sources → ◉ **Add or create constraints** → Next → Add Files → `Z1.xdc` → ☑ **Copy constraints files into project** → Finish.

Sposób z szablonu (tak uczy prowadzący):

1. Tak samo dodaj plik `ZU3.xdc` (szablon ze strony przedmiotu).
2. W Sources → Constraints → constrs_1 kliknij 2× na `ZU3.xdc`.
3. Znajdź sekcję **Slide Switches**. W liniach z `PL_USER_SW[0]` … `[3]` oraz w linii `IOSTANDARD … PL_USER_SW*`:
   - usuń znak `#` z początku linii,
   - zamień `PL_USER_SW` na `X`.
4. To samo w sekcji **White LEDS**: linie `PL_USER_LED[0]` … `[3]` + linia IOSTANDARD, zamień na `Y`.
5. Ctrl+S. Końcowy efekt ma wyglądać tak:

```
## Z1: X(3:0) <- PL_USER_SW(3:0), Y(3:0) -> PL_USER_LED(3:0)
## Piny z master XDC AUP-ZU3 (Real Digital); SW zgodne z Vivado_Instrukcja.pdf 4.1.
## Na zajeciach i tak sprawdz z szablonem ZU3.xdc ze strony przedmiotu.

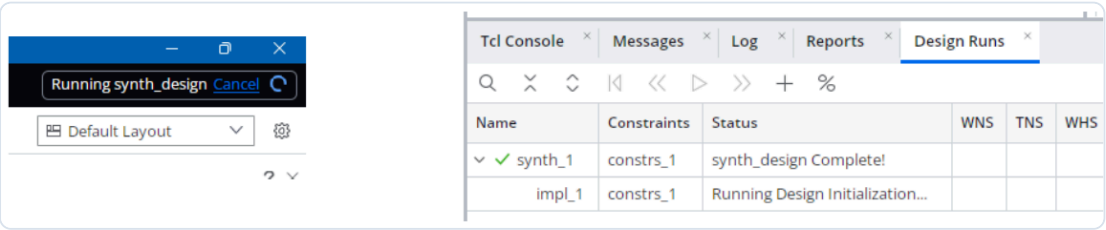
set_property PACKAGE_PIN AB1 [get_ports {X[0]}]
set_property PACKAGE_PIN AF1 [get_ports {X[1]}]
set_property PACKAGE_PIN AE3 [get_ports {X[2]}]
set_property PACKAGE_PIN AC2 [get_ports {X[3]}]
set_property IOSTANDARD LVCMOS12 [get_ports X*]

set_property PACKAGE_PIN AF5 [get_ports {Y[0]}]
set_property PACKAGE_PIN AE7 [get_ports {Y[1]}]
set_property PACKAGE_PIN AH2 [get_ports {Y[2]}]
set_property PACKAGE_PIN AE5 [get_ports {Y[3]}]
set_property IOSTANDARD LVCMOS12 [get_ports Y*]
```

> ⚠ **Najczęstszy błąd**
> Zapomniana linia `IOSTANDARD` albo literówka w nazwie (`X` w XDC musi być identyczne jak port `X` w VHDL). Pozostałych sekcji szablonu nie ruszaj — zostają z `#`.

11 Zbuduj plik dla płyty (bitstream)

1. Flow Navigator → PROGRAM AND DEBUG → Generate Bitstream.
2. Pytanie „No implementation results available… launch synthesis and implementation?" → Yes.
3. Okno „Launch Runs" → OK.
4. **Czekaj** (kilka minut). Postęp widać w prawym górnym rogu okna („Running synth_design…").
5. Na końcu pojawi się okno „Bitstream Generation Completed" → wybierz ◉ **Open Hardware Manager** → OK.

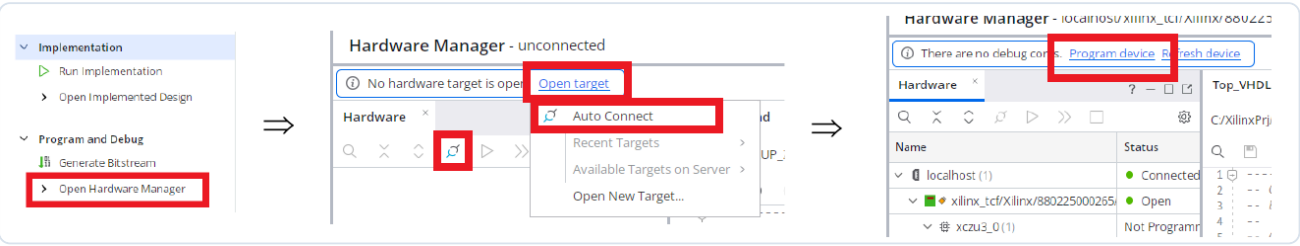


Postęp budowania: prawy górny róg i zakładka Design Runs na dole

✔ SPRAWDŹ Prawy górny róg: **write_bitstream Complete** ✓. Czerwone błędy? → zakładka **Messages** na dole i rozdział „Coś nie działa”.

12 Wgraj do płyty i przetestuj

1. Upewnij się, że płyta jest podłączona kablem USB do komputera i **włączona**.
2. W Hardware Manager kliknij na zielonym pasku u góry **Open target** → **Auto Connect**.
3. Kliknij **Program device**. W okienku pole **Bitstream file** powinno wskazywać `…/Z1.runs/impl_1/Z1.bit`. Jeśli jest pusto lub inna ścieżka — kliknij „…”, wejdź do folderu projektu → `Z1.runs` → `impl_1` → `Z1.bit`.
4. **Program**. Po kilku sekundach płyta działa.



Hardware Manager: Open target → Auto Connect → Program device

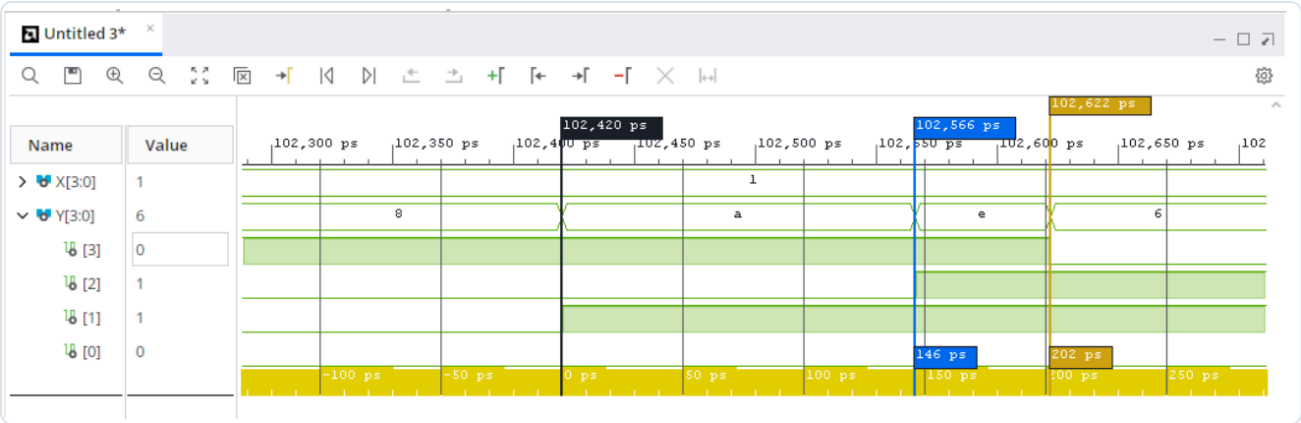
✔ SPRAWDŹ (test na płycie) Przełączniki **SW3 SW2 SW1 SW0** = X, diody **LED3 LED2 LED1 LED0** = Y. Przykład wariantu a): SW = 0101 (5) → LED = 0110 (6). Przetestuj kilka wartości, m.in. X = 15.

⚠ **Zmieniłeś kod?**
Musisz znowu kliknąć **Generate Bitstream** i znowu **Program device**. Płyta sama się nie zaktualizuje.

13 Symulacja czasowa (prawdziwe opóźnienia)

Teraz Vivado symuluje **gotowy sprzęt** po implementacji — widać, ile ns zajmuje przejście sygnału przez układ.

1. Flow Navigator → SIMULATION → **Run Simulation** → **Run Post-Implementation Timing Simulation**.
2. Interesuje nas zmiana **X: 0 → 1**, czyli moment **100 ns**. Powiększaj okolice 100 ns (**Zoom In** / `Ctrl`+kółko myszy).
3. Kliknij strzałkę ▸ obok **Y** — rozwinie się na pojedyncze bity Y[3] … Y[0].
4. Pomiar opóźnienia: kliknij na zbocze X → **Add Marker** → zaznacz bit Y → **Next Transition**. Vivado pokaże różnicę czasu.



Przykład z instrukcji: rozwinięte bity Y i markery mierzące opóźnienie

Co powiedzieć prowadzącemu

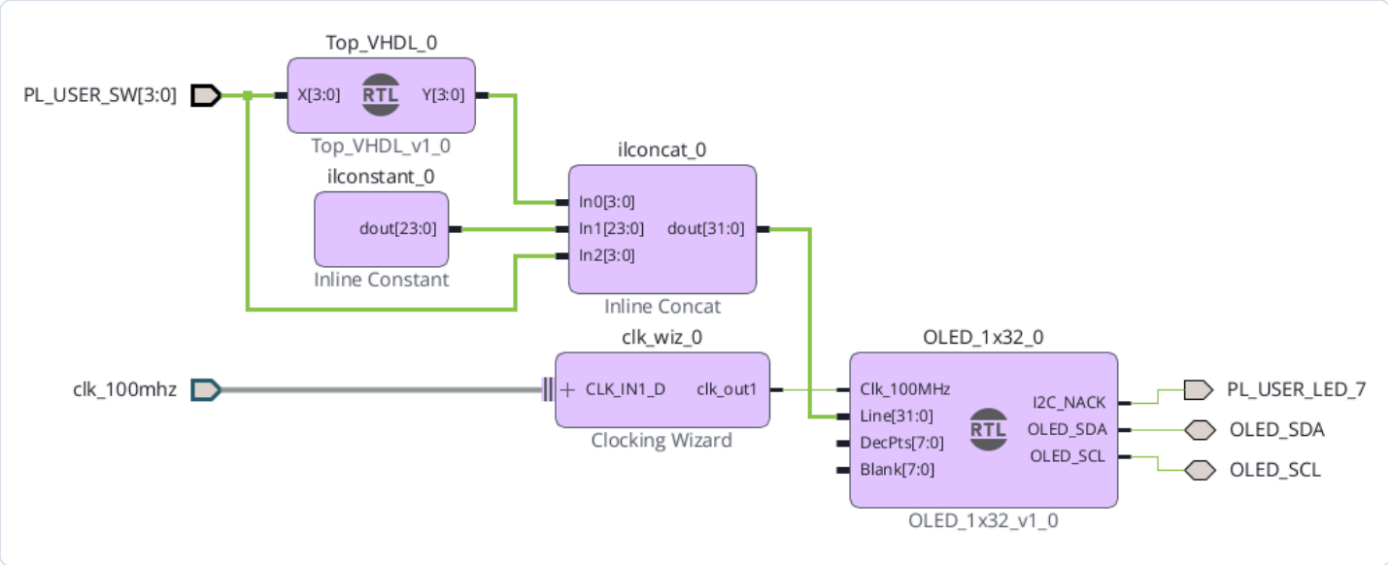
- Y zmienia się dopiero **kilka ns po** X — to opóźnienie bramek, połączeń i buforów wejścia/wyjścia.
- Bity Y nie zmieniają się dokładnie w tej samej chwili, więc przez ułamek ns na Y widać **fałszywą wartość** (tzw. hazard / glitch).
- Dla zmiany X: 0 → 1 zmieniają się bity: a) Y0, Y1 · b) Y0, Y1, Y2 · c) wszystkie 4 · d) Y0, Y1. Im więcej bitów się zmienia, tym więcej możliwych „śmieciowych” stanów pośrednich.

✔ ZADANIE 1 GOTOWE, GDY POKAZAŁEŚ (1) równania z map Karnaugha, (2) symulację behawioralną 0…15, (3) działanie na płycie, (4) symulację czasową dla X: 0 → 1.

Zadanie 2 · sposób A: diagram blokowy

Tak jak w instrukcji prowadzącego (rozdz. 5): klocki łączone myszką. X z przełączników, X i Y na wyświetlaczu OLED.

Docelowo diagram ma wyglądać tak (zrzut z instrukcji prowadzącego — u Ciebie zamiast `Top_VHDL_0` będzie `Z1_0`):



Cel: PL_USER_SW → Z1 → Concat → OLED, zegar przez Clocking Wizard

Jak to działa

OLED pokazuje 8 cyfr szesnastkowych (0–F). Każda cyfra to 4 bity wektora `Line[31:0]`. Klocek **Concat** skleja: **X** (4 bity, lewa cyfra) + **24 zera** (6 środkowych cyfr) + **Y** (4 bity, prawa cyfra). Wynik np. dla wariantu a) i X = 9: `9000000A` (A = 10 szesnastkowo).

1 Nowy projekt Z2

Zrób **nowy projekt** dokładnie jak w Zadaniu 1, kroki 1–5, ale nazwij go `Z2`.

> 💡 **Dlaczego nowy?**
> Stary plik XDC z Zadania 1 (porty X, Y) przeszkadzałby tutaj. Czysty projekt = mniej problemów.

2 Dodaj pliki: Twój Z1 i moduł OLED

1. **Add Sources** → ◉ **Add or create design sources** → **Next** → **Add Files**.
2. Wybierz:
   - swój plik `Z1` — z poprzedniego projektu: `C:/AMD_Proj/Z1/Z1.srcs/sources_1/new/Z1.vhd` (albo `Z1_x.vhd` z pendrive'a),
   - `OLED_1x32.vhd` (od prowadzącego).
3. Zaznacz ☑ **Copy sources into project** → **Finish**.

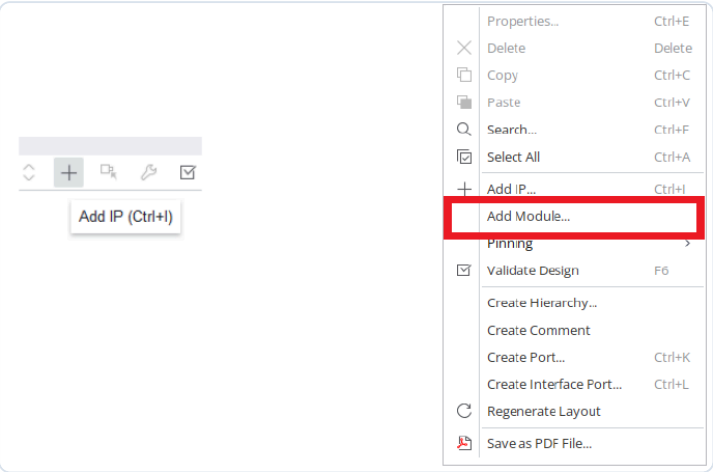
✔ **SPRAWDŹ** W Design Sources są **Z1** i **OLED_1x32**.

3 Utwórz pusty diagram

1. Flow Navigator → IP INTEGRATOR → **Create Block Design**.
2. Design name: `Z2_BD` → **OK**. Otworzy się pusta, biała plansza (Diagram).

4 Wstaw swoje moduły VHDL (Z1 i OLED)

1. PPM na pustym miejscu planszy → **Add Module...** → wybierz **Z1** → **OK**.
2. Jeszcze raz: PPM → **Add Module...** → **OLED_1x32** → **OK**.



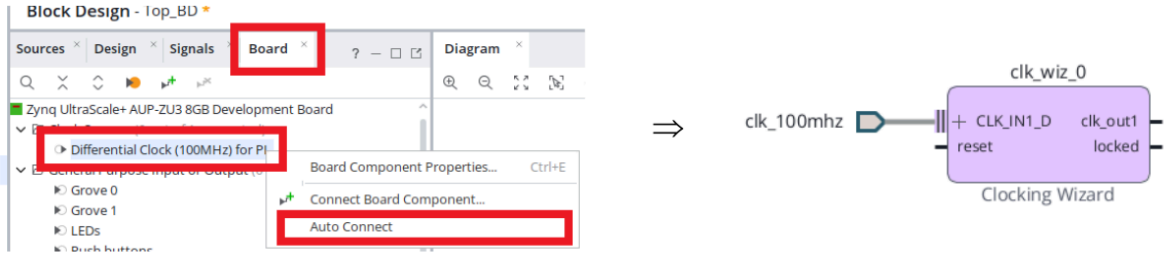
Menu po PPM na planszy: Add IP (biblioteka Vivado) i Add Module (Twoje pliki VHDL)

✔ **SPRAWDŹ** Na planszy są dwa fioletowe klocki z napisem **RTL**: **Z1_0** i **OLED_1x32_0**.

5 Zegar 100 MHz (Clocking Wizard)

OLED potrzebuje zegara. Płyta ma zegar 100 MHz, a Vivado podłączy go sam:

1. W panelu po lewej (obok Sources / Design / Signals) kliknij zakładkę **Board**.
2. Znajdź pozycję **Differential Clock (100MHz) for PL** (rozwijaj gałęzie strzałkami ▸).
3. PPM na niej → **Auto Connect**.



Board → Differential Clock → Auto Connect = pojawia się clk_wiz_0 z portem clk_100mhz

Wyłącz zbędne wejście reset i wyjście locked: kliknij 2× na klocek **clk_wiz_0** → zakładka **Output Clocks** → przewiń na sam dół → odznacz ☐ **reset** i ☐ **locked** → **OK**.

Nie możesz znaleźć tych opcji? Zostaw — niepodłączony reset i tak ma wartość 0, a układ zadziała.

> ⚠ **Zapamiętaj**
> Przy Clocking Wizard w pliku XDC **nie** odkomentowujesz linii `CLK_100MHz_*` — wizard robi to sam.

6 Klocek „stała” (24 zera)

1. Kliknij `+` (Add IP) na pasku planszy albo `Ctrl`+`I`.
2. W wyszukiwarce wpisz `constant` → kliknij 2× na **Inline Constant** (lub „Constant”).
3. Kliknij 2× na nowy klocek. Ustaw: **Const Width** = `24`, **Const Val** = `0` → **OK**.

7 Klocek „sklejacz” (Concat)

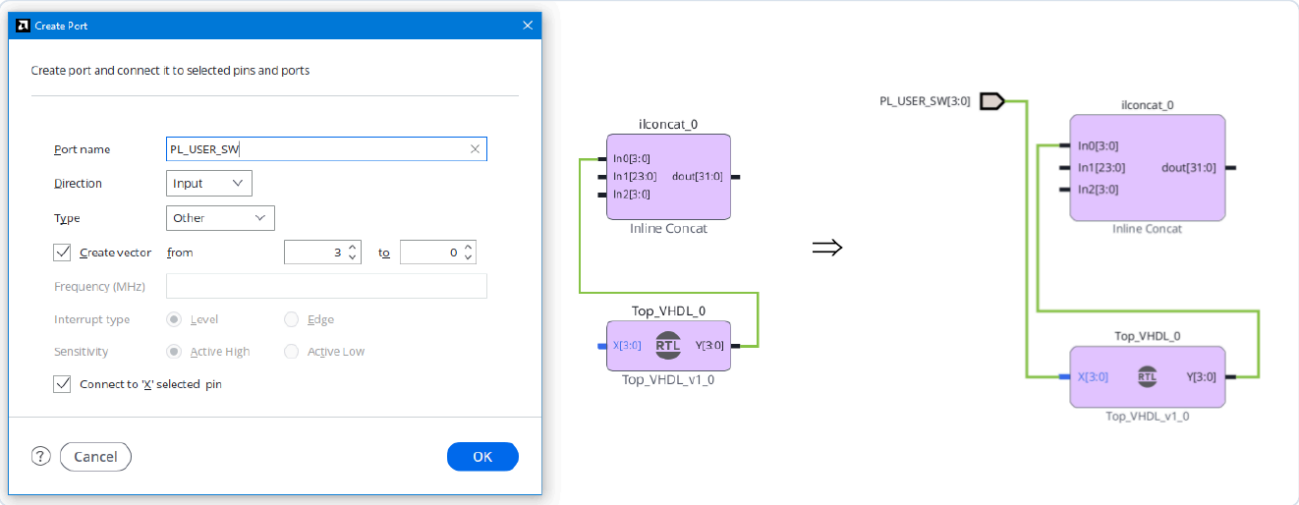
1. `+` Add IP → wpisz `concat` → 2× na **Inline Concat** (lub „Concat”).
2. Kliknij 2× na klocek i ustaw: **Number of Ports** = `3`, **In0 Width** = `4`, **In1 Width** = `24`, **In2 Width** = `4` → **OK**.

Nazwy pól mogą się lekko różnić w zależności od wersji Vivado — chodzi o 3 wejścia o szerokościach 4, 24, 4.

> **Która cyfra gdzie?**
> **In0** = najmłodsze bity = **prawa** cyfra (Y). **In2** = najstarsze bity = **lewa** cyfra (X).

8 Port dla przełączników

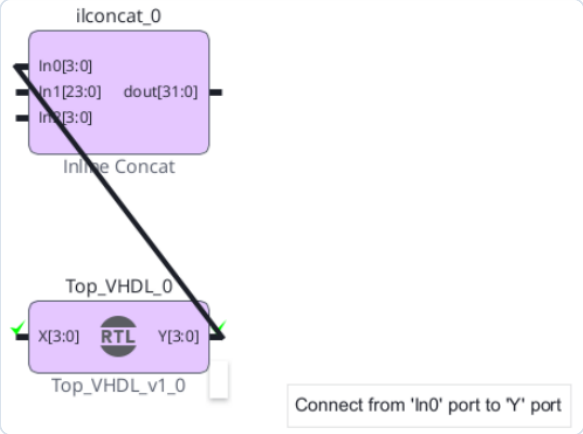
1. PPM na nóżce **X[3:0]** klocka Z1_0 → **Create Port...** (`Ctrl`+`K`).
2. Ustaw: Port name = `PL_USER_SW`, Direction = **Input**, Type = **Other**, ☑ **Create vector** from `3` to `0`, ☑ **Connect to selected pin** → **OK**.



Create Port: nazwa PL_USER_SW, wektor 3..0 — port od razu łączy się z X

9 Połącz klocki myszką

Jak rysować połączenie: najedź na koniec nóżki (kursor zmieni się w ołówek), **przytrzymaj lewy przycisk**, przeciągnij do drugiej nóżki, puść.



Przeciąganie połączenia od nóżki do nóżki

#	Od	Do	Po co
1	port **PL_USER_SW[3:0]** (albo nóżka X[3:0])	**In2[3:0]** w Concat	X na lewej cyfrze
2	**Y[3:0]** w Z1_0	**In0[3:0]** w Concat	Y na prawej cyfrze
3	**dout[23:0]** w Constant	**In1[23:0]** w Concat	zera w środku
4	**dout[31:0]** w Concat	**Line[31:0]** w OLED_1x32_0	co wyświetlić
5	**clk_out1** w clk_wiz_0	**Clk_100MHz** w OLED_1x32_0	zegar

Pomyłka? PPM na nóżce → **Disconnect Pin** i narysuj jeszcze raz.

10 Wyprowadź OLED i sygnał błędu na zewnątrz

1. PPM na nóżce **OLED_SDA** klocka OLED → **Make External** (`Ctrl`+`T`). To samo dla **OLED_SCL** i **I2C_NACK**.
2. Vivado nazwie porty np. `OLED_SDA_0`. **Zmień nazwy**: kliknij port (strzałkę po prawej) → w okienku **External Port Properties** (lewy dół) w polu **Name** wpisz:
   - `OLED_SDA_0` → `OLED_SDA`
   - `OLED_SCL_0` → `OLED_SCL`
   - `I2C_NACK_0` → `NACK`

> ⚠ **Nazwy muszą się zgadzać z XDC**
> Plik `Z2_BD.xdc` szuka portów: `PL_USER_SW`, `OLED_SDA`, `OLED_SCL`, `NACK`.

11 (Opcjonalnie) Ukryj zera w środku wyświetlacza

Bez tego OLED pokaże np. `9000000A`. Żeby było `9 A`:

1. `+` Add IP → **Inline Constant** → 2× klik: Const Width = `8`, Const Val = `126` (binarnie 0111 1110 = wygaś 6 środkowych cyfr).
2. Połącz jego **dout[7:0]** z **Blank[7:0]** w OLED_1x32_0.

Wejście **DecPts** (kropki) możesz zostawić niepodłączone — tak jest w przykładzie prowadzącego.

12 Uporządkuj, sprawdź, zapisz

1. Kliknij **Regenerate Layout** (⟳ na pasku planszy) — Vivado ładnie poukłada klocki.

2. Kliknij **Validate Design** (ikona ☑ na pasku albo **F6**).
3. Zapisz: **Ctrl**+**S**.



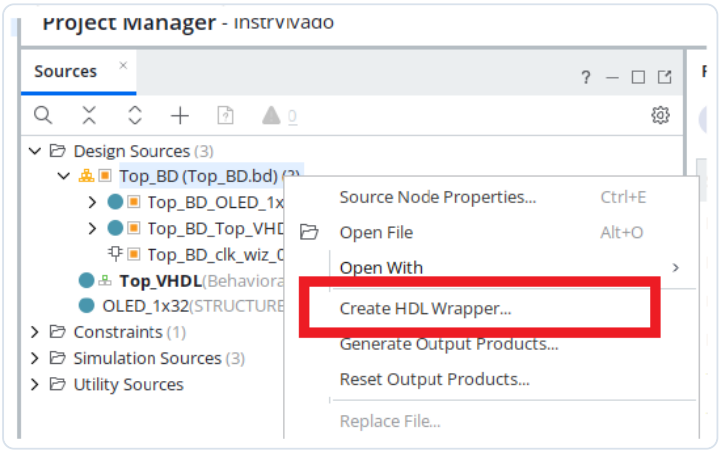
Regenerate Layout porządkuje diagram

✔ **SPRAWDŹ** Validate: „Validation successful”. Ostrzeżenia (żółte) o niepodłączonym DecPts są OK. Błędy (czerwone) — przeczytaj, zwykle chodzi o niepołączoną nóżkę.

13 Opakowanie diagramu (HDL Wrapper)

Diagram nie może sam być „top”. Trzeba go opakować w plik VHDL:

1. Wróć do okna **Sources**. W Design Sources PPM na **Z2_BD** (plik `.bd`) → **Create HDL Wrapper...**.
2. Wybierz ◉ **Let Vivado manage wrapper and auto-update** → **OK**.



PPM na diagramie w Sources → Create HDL Wrapper

✔ **SPRAWDŹ** Na górze Design Sources jest **Z2_BD_wrapper** pogrubiony (top). Jeśli nie: PPM → **Set as Top**.

14 Plik XDC dla Z2

Sposób łatwy: **Add Sources** → constraints → **Add Files** → `Z2_BD.xdc` → ☑ Copy → **Finish**. Jego zawartość:

```
## Z2 (wersja z diagramem blokowym): nazwy portow jak w diagramie.
## Zegara CLK_100MHz_* tu NIE opisujemy -- robi to Clocking Wizard.

## Przelaczniki SW(3:0) -> port PL_USER_SW[3:0]
set_property PACKAGE_PIN AB1 [get_ports {PL_USER_SW[0]}]
set_property PACKAGE_PIN AF1 [get_ports {PL_USER_SW[1]}]
set_property PACKAGE_PIN AE3 [get_ports {PL_USER_SW[2]}]
set_property PACKAGE_PIN AC2 [get_ports {PL_USER_SW[3]}]
set_property IOSTANDARD LVCMOS12 [get_ports PL_USER_SW*]

## LED7 <- NACK (swieci = blad komunikacji z OLED)
set_property PACKAGE_PIN AF2 [get_ports {NACK}]
set_property IOSTANDARD LVCMOS12 [get_ports NACK]

## OLED IIC (plyta rozszerzajaca)
set_property PACKAGE_PIN AE12 [get_ports {OLED_SCL}]
set_property PACKAGE_PIN AF12 [get_ports {OLED_SDA}]
set_property IOSTANDARD LVCMOS33 [get_ports OLED_*]
```

Sposób z szablonów: dodaj `ZU3.xdc` i `ZU3_Exp.xdc`, potem:

- `ZU3.xdc`: odkomentuj (usuń `#`) linie `PL_USER_SW[0]` … `[3]` + ich IOSTANDARD — nazw **nie zmieniaj**, port nazywa się tak samo.
- `ZU3.xdc`: odkomentuj linię `PL_USER_LED[7]` (pin AF2) i zmień nazwę na `NACK`; dopisz `set_property IOSTANDARD LVCMOS12 [get_ports NACK]`.
- `ZU3_Exp.xdc`: odkomentuj 3 linie sekcji **OLED IIC**.
- **Nie** odkomentowuj linii zegara `CLK_100MHz_*`.

15 Bitstream, wgranie, test

1. **Generate Bitstream** → **Yes** → **OK**, czekaj (dłużej niż w Z1 — Vivado buduje też Clocking Wizard).
2. Open Hardware Manager → **Open target** → **Auto Connect** → **Program device** → plik `Z2_BD_wrapper.bit` → **Program**.

✔ SPRAWDŹ OLED (płyta rozszerzająca) pokazuje: **lewa cyfra = X**, **prawa = Y**, szesnastkowo (10 = A … 15 = F). Ruszaj SW3…SW0 — wartości zmieniają się od razu. LED7 **nie** świeci.

⚠ **LED7 świeci / OLED pusty?**
Błąd komunikacji z OLED: sprawdź, czy płyta rozszerzająca jest wpięta, i czy w XDC są linie OLED z `LVCMOS33`.

Zadanie 2 · sposób B: gotowy kod + tryb AUTO

Bez rysowania diagramu. Dodatkowo X może zmieniać się sam (odpowiedź na „zmianę co 100 ns”).

Skąd „co 100 ns”?

W poleceniu Zadania 1 „co 100 ns” dotyczy **testbencha** (symulacji). Na płycie zmiana co 100 ns to 10 milionów zmian na sekundę — OLED odświeża się ~50 razy na sekundę, więc nic byś nie zobaczył. Dlatego `Z2_Top.vhd` ma parametr `STEP_CYCLES` (liczba taktów zegara 100 MHz na jeden krok):

- `STEP_CYCLES = 10` → X zmienia się co **100 ns** — ustawione w testbenchu `Z2_tb.vhd`,
- `STEP_CYCLES = 100E6` (domyślnie) → co **1 s** na płycie — widać gołym okiem.

Element płyty	Funkcja
SW3…SW0	X w trybie ręcznym
SW7	0 = tryb ręczny, 1 = **AUTO** (X liczy sam 0 → 15 → 0 …)
LED7…LED4 / LED3…LED0	X / Y binarnie
OLED	X i Y **dziesiętnie**, np. `13 14`

1 Nowy projekt i pliki

1. Nowy projekt jak w Z1 (kroki 1–5), nazwa np. `Z2b`.
2. **Add Sources** → design sources → **Add Files**: `Z1_x.vhd` (Twój wariant), `OLED_1x32.vhd`, `Z2_Top.vhd` → ☑ Copy → **Finish**.
3. **Add Sources** → simulation sources → **Add Files**: `Z2_tb.vhd` → **Finish**.
4. **Add Sources** → constraints → **Add Files**: `Z2.xdc` → **Finish**.
5. W Design Sources PPM na **Z2_Top** → **Set as Top** (jeśli nie jest pogrubiony).

2 Ustaw wariant w testbenchu

Otwórz `Z2_tb.vhd` i w linii `Generic ( K : integer := 1 );` wpisz swój wariant: a) `1`, b) `3`, c) `-1`, d) `-3`. **Ctrl**+**S**.

K służy tylko do automatycznego sprawdzania wyników w symulacji.

3 Symulacja (X co 100 ns)

1. **Run Behavioral Simulation**, potem w Tcl Console: `run 2500 ns`, **Zoom Fit**.
2. Radix → Unsigned Decimal na sygnale **LED** (albo rozwiń go strzałką ▸).

✔ SPRAWDŹ Od ok. 200 ns (SW7 = 1) X w LED[7:4] zmienia się co 100 ns, a LED[3:0] = Y. W Tcl Console jest „Koniec testu” i **brak** komunikatów „Blad…”.

Symulacja zawiera prawdziwy moduł OLED, więc może trwać chwilę dłużej.

4 Bitstream, wgranie, test

Generate Bitstream → **Program device** (plik `Z2_Top.bit`) — tak samo jak w Z1.

✔ SPRAWDŹ SW7 = 0: OLED pokazuje X z przełączników i Y. SW7 = 1: liczby zmieniają się same co 1 s.

Szybciej / wolniej na płycie?
W `Z2_Top.vhd` zmień `STEP_CYCLES : positive := 100E6` np. na `50E6` (0,5 s) → zbuduj bitstream od nowa.

⚠ **X i Y zamienione miejscami na OLED?**
W `Z2_Top.vhd` zamień kolejność: `Line <= DY & X"0000" & DX;`

Coś nie działa — najczęstsze problemy

Objaw	Co zrobić
Nie ma płyty AUP-ZU3 na liście Boards	Zakładka **Boards** (nie Parts), wyszukaj „AUP”. Dalej brak → zawołaj prowadzącego (brak definicji płyty w Vivado).
Czerwona kreska / podkreślenie w kodzie	Najedź na czerwoną kreskę na prawym brzegu edytora i przeczytaj błąd. Najczęściej: brak `;`, literówka, `A and B or C` bez nawiasów (musi być `(A and B) or C`).
Symulacja nie startuje, błąd kompilacji	Zakładka **Messages** → pierwszy czerwony błąd, kliknij link do pliku. Nazwa w `component Z1` musi być identyczna jak encja `Z1`.
Na wykresie tylko X = 0 … 9	Symulacja trwa 1000 ns. Tcl Console: `run 600 ns`.
Y jest czerwone / „U” / „X”	Testbench nie widzi układu: sprawdź hierarchię (Z1_tb → UUT : Z1) i **Set as Top** na Z1_tb.
Poprawiłem kod, a symulacja pokazuje stare wyniki	`Ctrl`+`S`, potem **Relaunch** (⟳) na pasku symulacji.
Bitstream: błąd `UCIO-1` lub `NSTD-1`	Któryś port nie ma pinu (`PACKAGE_PIN`) albo standardu (`IOSTANDARD`) w XDC. Sprawdź, czy odkomentowałeś obie linie.
Critical Warning: „No ports matched 'X[0]'”	Nazwa portu w XDC inna niż w VHDL/diagramie (np. `OLED_SDA_0` zamiast `OLED_SDA`). Popraw nazwę w XDC albo zmień nazwę portu.
Hardware Manager: „No hardware target”	Sprawdź kabel USB i zasilanie płyty, kliknij jeszcze raz **Open target** → **Auto Connect**.
Płyta działa „po staremu”	Po każdej zmianie: Generate Bitstream + Program device. W oknie wyboru pliku sprawdź datę `.bit` (okno pamięta folder z innego projektu!).
Diagram: „block design cannot be top”	Brak wrappera: PPM na `.bd` → Create HDL Wrapper (krok 13 sposobu A).
OLED nic nie pokazuje	Płyta rozszerzająca wpięta? W XDC linie OLED odkomentowane, `LVCMOS33` ? Zegar podłączony do `Clk_100MHz` ? Świeci LED7 (NACK) = brak odpowiedzi OLED.
Wszystko się posypało	Tcl Console: `reset_project` , potem Generate Bitstream od nowa.

> 💡 **Złota zasada**
> Po każdej zmianie w kodzie: `Ctrl`+`S` → (symulacja: Relaunch) → (płyta: Generate Bitstream → Program device).